

Information Retrieval with PyLucene

Study Oriented Project (SOP) Final Report

Covering IR Foundations, Ranking Models, and
PyLucene Indexing & Retrieval in Depth

Submitted by:

Aayush Vij & Vandan

Submitted to:

Prof. Abhisek Chakrabarty

Repository github.com/aayushvij04/IR
Date April 29, 2026

Source code and this report are available in the linked repository.

Information Retrieval — Foundations

Information Retrieval (IR) is the science of finding documents relevant to an information need from large collections. Unlike database retrieval, IR operates on unstructured text and copes with ambiguity and redundancy. A classical IR system has three pillars:

1. **Corpus and Documents.** A collection of textual units over which retrieval is performed.
2. **Query Processing.** Translating an information need into Boolean expressions, vectors, or probabilities.
3. **Ranking Model.** A mathematical model that assigns a relevance score to each document.

The Inverted Index

The *inverted index* is the foundational data structure. For every vocabulary term t , it stores a *postings list*—a sorted list of document IDs:

$$\text{Index} : t \longrightarrow \langle df_t, [d_1, d_2, \dots, d_{df_t}] \rangle \quad (1)$$

Boolean AND queries are answered in $O(|p_1| + |p_2|)$ via a linear merge of sorted lists, accelerated with *skip pointers* every $\lfloor \sqrt{n} \rfloor$ positions. Construction at scale uses **BSBI** (Block Sort-Based Indexing) or **SPIMI** (Single-Pass In-Memory Indexing). Postings are compressed with Variable-Byte or Gamma encoding to reduce footprint, motivated by Zipf’s Law and Heaps’ Law ($M \approx k n^b$).

Ranking Models

Vector Space Model (VSM). Documents and queries are TF-IDF weighted vectors in $\mathbb{R}^{|V|}$. Relevance is the *cosine similarity*:

$$\text{sim}(q, d) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| |\vec{d}|} \quad \text{where} \quad w_{t,d} = (1 + \log \text{tf}_{t,d}) \cdot \log \frac{N}{df_t} \quad (2)$$

BM25 (Okapi). The probabilistic ranking function dominating modern search engines:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f_{t,d}(k_1 + 1)}{f_{t,d} + k_1(1 - b + b \frac{|d|}{\text{avdl}})} \quad (3)$$

Typical parameters are $k_1 \in [1.2, 2.0]$ and $b = 0.75$. Extensions include **BM25F** (field-weighted) and **BM25+** (lower-bound on TF saturation).

Language Models (LM). Query likelihood is $P(q | d) = \prod_{t \in q} P(t | d)$. The zero-frequency problem is solved with smoothing methods such as *Jelinek-Mercer*: $P_\lambda(t | d) = \lambda P(t | d)_{\text{MLE}} + (1 - \lambda)P(t | C)$, or *Dirichlet*: $P_\mu(t | d) = \frac{f_{t,d} + \mu P(t|C)}{|d| + \mu}$.

Evaluation

Retrieval quality is measured by **MAP** (Mean Average Precision), **nDCG@K** (Normalized Discounted Cumulative Gain), and **MRR** (Mean Reciprocal Rank). Evaluation often follows the **TREC** paradigm using pooled judgements and `trec_eval`. Inter-annotator agreement is quantified by Cohen’s κ .

PyLucene — Architecture and Setup

Apache Lucene is a high-performance, open-source full-text search library written in Java. **PyLucene** exposes the full Lucene API to Python via JCC, a C++ code generator that bridges the JVM and CPython runtimes. This means every Java class in Lucene 9.x is directly accessible as a Python class—with identical semantics and performance.

Initializing the JVM

Every PyLucene session must start the Java Virtual Machine exactly once:

```
1 import lucene
2 lucene.initVM(vmargs=['-Djava.awt.headless=true'])
```

Listing 1: JVM initialization

Storage Backends

```
1 from org.apache.lucene.store import FSDirectory, ByteBuffersDirectory
2 from java.nio.file import Paths
3
4 # Persistent on-disk index
5 directory = FSDirectory.open(Paths.get("/path/to/index"))
6
7 # Ephemeral in-memory index (testing / unit tests)
8 directory = ByteBuffersDirectory()
```

Listing 2: On-disk vs in-memory index

Indexing with PyLucene

IndexWriter and Analyzers

```
1 from org.apache.lucene.analysis.standard import StandardAnalyzer
2 from org.apache.lucene.index import IndexWriter, IndexWriterConfig
3
4 analyzer = StandardAnalyzer() # lowercase + stop-word removal
5 config = IndexWriterConfig(analyzer)
6 config.setOpenMode(IndexWriterConfig.OpenMode.CREATE)
7 writer = IndexWriter(directory, config)
```

Listing 3: Creating an IndexWriter with StandardAnalyzer

Available analyzers and their behaviours are summarized in Table 1.

Analyzer	Tokenization	Typical Use
StandardAnalyzer	Unicode tokenizer + lowercase + stop words	General English text
WhitespaceAnalyzer	Split on whitespace, preserves case	Code, identifiers
KeywordAnalyzer	Entire field = one token	IDs, enums, genres
EnglishAnalyzer	StandardAnalyzer + Porter stemming	IR experiments
CJKAnalyzer	Bi-gram tokenization	Chinese/Japanese/Korean

Table 1: PyLucene Analyzer options

Document and Field Types

A Lucene Document is a flat set of Field objects. Field type controls *whether* and *how* a value is indexed:

```
1 from org.apache.lucene.document import (
```

```

2 Document, TextField, StringField,
3   StoredField, IntPoint, FloatPoint, Field)
4
5 def movie_to_document(movie: dict) -> Document:
6     doc = Document()
7     # Analyzed full-text - tokenized, lowercased, stop-word removed
8     doc.add(TextField("title", movie["title"], Field.Store.YES))
9     doc.add(TextField("plot", movie["plot"], Field.Store.YES))
10    # Exact keyword - single token, not analyzed
11    doc.add(StringField("genre", movie["genre"], Field.Store.YES))
12    # Stored only - retrieve at search time, not searchable
13    doc.add(StoredField("year_display", str(movie["year"])))
14    # Numeric range-queryable fields
15    doc.add(IntPoint("year", movie["year"]))
16    doc.add(FloatPoint("rating", movie["rating"]))
17    doc.add(StoredField("year", movie["year"]))
18    doc.add(StoredField("rating", movie["rating"]))
19    return doc

```

Listing 4: Field types in a movie document (from `src/pylucene/indexing.py`)

Field Type	Indexed	Analyzed	Stored
TextField	Yes	Yes	Optional
StringField	Yes	No	Optional
StoredField	No	No	Yes
IntPoint	Yes (BKD tree)	No	No
FloatPoint	Yes (BKD tree)	No	No

Table 2: Lucene field type comparison

Retrieval with PyLucene

IndexSearcher and Similarity

```

1 from org.apache.lucene.index import DirectoryReader
2 from org.apache.lucene.search import IndexSearcher
3 from org.apache.lucene.search.similarities import BM25Similarity
4
5 reader = DirectoryReader.open(directory)
6 searcher = IndexSearcher(reader)
7 searcher.setSimilarity(BM25Similarity(1.2, 0.75)) # k1=1.2, b=0.75
8
9 from org.apache.lucene.queryparser.classic import QueryParser
10 parser = QueryParser("plot", analyzer)
11 query = parser.parse("space travel wormhole")
12 top_docs = searcher.search(query, 5) # top-5 results
13
14 for hit in top_docs.scoreDocs:
15     doc = searcher.doc(hit.doc)
16     print(f"Score={hit.score:.4f} | {doc.get('title')}")
17 reader.close()

```

Listing 5: Setting up IndexSearcher with BM25 (from `src/pylucene/retrieval.py`)

Explaining Scores

`IndexSearcher.explain()` returns a human-readable breakdown of how each relevance score was computed—invaluable for debugging:

```

1 hit = top_docs.scoreDocs[0]

```

```

2 explanation = searcher.explain(query, hit.doc)
3 print(explanation.toString())
4 # Output includes: IDF, TF, field norm, BM25 saturation factor...

```

Listing 6: Score explanation

PyLucene Query Classes

PyLucene provides a rich query API. All examples below are drawn from `src/pylucene/query_classes.py` in the repository.

TermQuery

Matches documents containing an *exact* post-analysis term:

```

1 from org.apache.lucene.search import TermQuery
2 from org.apache.lucene.index import Term
3
4 query = TermQuery(Term("plot", "crime"))
5 top_docs = searcher.search(query, 5)

```

Listing 7: TermQuery – exact single-term match

PhraseQuery

Ordered phrase match with optional `slop` for proximity search:

```

1 from org.apache.lucene.search import PhraseQuery
2
3 # Exact phrase
4 exact = (PhraseQuery.Builder()
5         .add(Term("plot", "corporate"))
6         .add(Term("plot", "secrets"))
7         .build())
8
9 # Proximity: terms within 3 positions of each other
10 prox = (PhraseQuery.Builder()
11        .add(Term("plot", "crime"))
12        .add(Term("plot", "redemption"))
13        .setSlop(3).build())

```

Listing 8: PhraseQuery – exact phrase and proximity

BooleanQuery

Combine sub-queries with **MUST** (AND), **SHOULD** (OR), **MUST_NOT** (NOT):

```

1 from org.apache.lucene.search import BooleanQuery, BooleanClause
2
3 builder = BooleanQuery.Builder()
4 builder.add(TermQuery(Term("plot", "crime")),
5             BooleanClause.Occur.SHOULD)
6 builder.add(TermQuery(Term("plot", "redemption")),
7             BooleanClause.Occur.SHOULD)
8 builder.add(TermQuery(Term("genre", "action")),
9             BooleanClause.Occur.MUST_NOT)
10 query = builder.build()

```

Listing 9: BooleanQuery – combining clauses (MUST/SHOULD/MUST NOT)

Numeric Range Queries

```

1 from org.apache.lucene.document import IntPoint, FloatPoint
2 from java.lang import Float
3
4 # Movies from 1990 to 2000
5 year_q = IntPoint.newRangeQuery("year", 1990, 2000)
6
7 # Movies with rating >= 9.0
8 rating_q = FloatPoint.newRangeQuery("rating", 9.0, Float.MAX_VALUE)

```

Listing 10: IntPoint and FloatPoint – numeric range queries

WildcardQuery and FuzzyQuery

```

1 from org.apache.lucene.search import WildcardQuery, FuzzyQuery
2
3 # ? = any single char, * = zero or more chars
4 wild = WildcardQuery(Term("plot", "r?dempt*"))
5
6 # Levenshtein distance <= 2 (handles typos)
7 fuzzy = FuzzyQuery(Term("plot", "astronawts"), 2)

```

Listing 11: WildcardQuery and FuzzyQuery – approximate matching

PrefixQuery and TermRangeQuery

```

1 from org.apache.lucene.search import PrefixQuery, TermRangeQuery
2
3 prefix = PrefixQuery(Term("plot", "inter")) # plot:inter*
4 trange = TermRangeQuery.newStringRange(
5     "genre", "crime", "drama", True, True) # inclusive

```

Listing 12: PrefixQuery and TermRangeQuery

MatchAllDocsQuery and BoostQuery

```

1 from org.apache.lucene.search import MatchAllDocsQuery, BoostQuery
2
3 all_q = MatchAllDocsQuery() # returns every doc
4
5 title_q = TermQuery(Term("title", "redemption"))
6 boosted = BoostQuery(title_q, 3.0) # title match = 3x score

```

Listing 13: MatchAllDocsQuery and BoostQuery

Execution Highlights

Running the core Python implementations on our sample datasets validates the theoretical models:

- **Boolean & Compression:** The inverted index correctly resolves complex queries (index AND NOT boolean). Variable-Byte encoding successfully compressed a sample postings list down to 8 bytes. Heaps' law estimated a 44,000 term vocabulary for 1M tokens.
- **Ranking Models:** BM25 out-performs standard VSM on the test queries. The custom evaluation module reported a **MAP of 0.491**, **MRR of 0.500**, and **nDCG@5 of 0.960**.
- **Language Models:** Dirichlet Smoothing ($\mu = 2000$) efficiently mitigated zero-frequency terms, improving the query likelihood scores over basic Jelinek-Mercer.
- **Web Search:** PageRank converged in 27 iterations on the simulated web graph, and HITS correctly identified the primary authoritative nodes.

Repository Overview

The full source code is available at:

<https://github.com/aayushvij04/IR>

File	Contents
<code>src/pylucene/indexing.py</code>	IndexWriter, Document, Field types, Analyzer choices
<code>src/pylucene/retrieval.py</code>	IndexSearcher, BM25/TF-IDF, multi-field, score explain
<code>src/pylucene/query_classes.py</code>	All query types with runnable demos
<code>src/boolean_retrieval.py</code>	Inverted index, Boolean ops, skip pointers (pure Python)
<code>src/ranking_models.py</code>	TF-IDF, VSM, BM25, BM25+ (pure Python)
<code>src/language_models.py</code>	Unigram LM, JM & Dirichlet smoothing, KLD
<code>src/evaluation.py</code>	P, R, AP, MAP, nDCG, MRR, Kappa
<code>src/web_search.py</code>	PageRank, HITS, Shingling
<code>src/advanced_ir.py</code>	LSI, co-occurrence embeddings, Rocchio
<code>report/IR_Report.tex</code>	This report (LaTeX source)

Table 3: Repository file index

Conclusion

PyLucene brings the full power of Apache Lucene to the Python ecosystem without sacrificing performance. Its rich field model, analyzer pipeline, and query API make it suitable for both research prototyping and production search systems. The integration of BM25 as the default similarity function—combined with flexible numeric indexing, proximity queries, and fuzzy matching—makes PyLucene one of the most complete IR toolkits available.

The accompanying code in this repository demonstrates each concept with runnable, well-commented examples, bridging the gap between IR theory and practical implementation.

Source: github.com/aayushvij04/IR